



HTML, CSS, PHP BASICS – VARIABLES, LOOPS, ARRAYS AND EMBEDDING SQL

SERVER-SIDE WEB DEVELOPMENT - 5COSC012C.2

INTENDED LESSON OUTCOME

- At the end of the lesson students will be able to
 - Recap on HTML, CSS and SQL
 - Use PHP as a server-side scripting language;
 - For Basic programming
 - To use Arrays of records
 - To embed SQL to retrieve info from DB
 - To use WHILE loops

TODAY'S TOPICS

- Recap on HTML, CSS and SQL
- PHP Basic programming constructs – PHP Variables
- Arrays of records in PHP
- Embed SQL to retrieve info from DB using PHP
- WHILE loops in PHP

HTML - INTRODUCTION

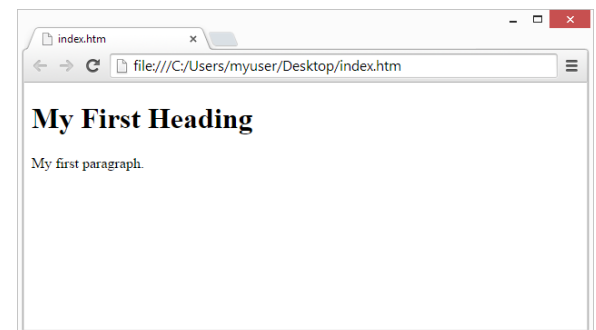
What is HTML?

- HTML is a markup language for describing web documents (web pages).
 - HTML stands for **H**yper **T**ext **M**arkup **L**anguage
 - A markup language is a set of markup tags
 - HTML documents are described by HTML tags
 - Each HTML tag describes different document content
 - HTML Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

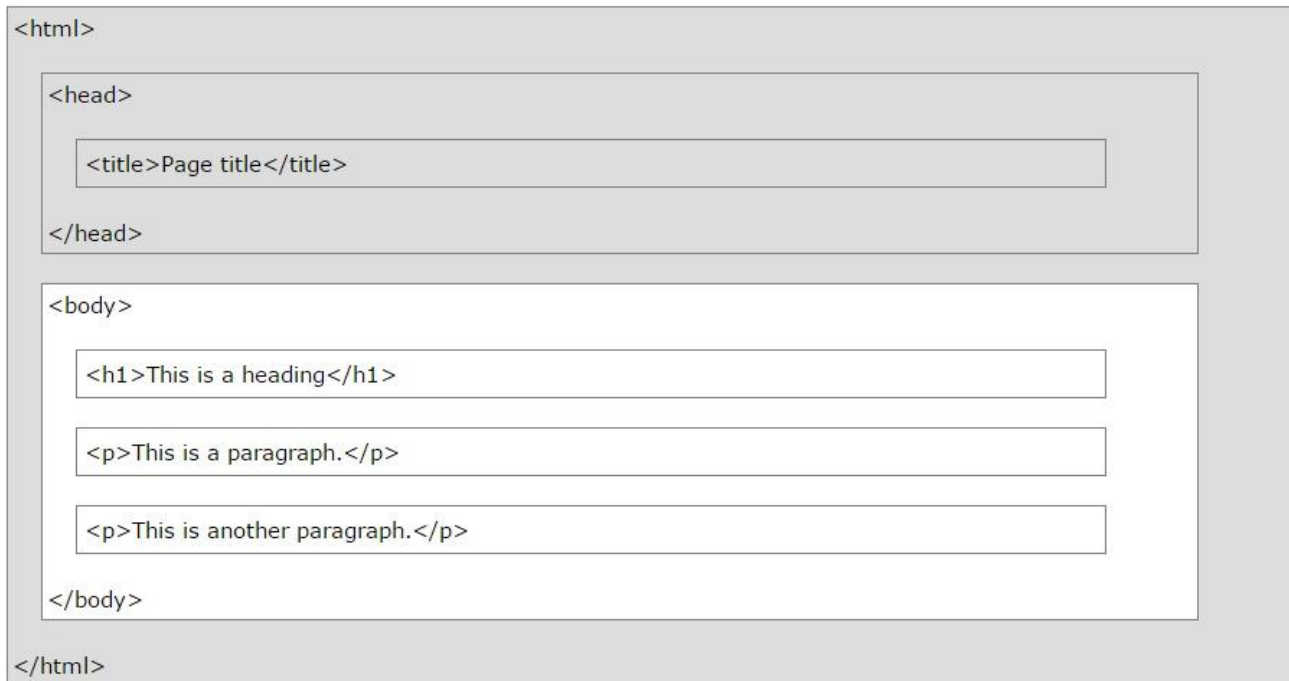
<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```



HTML PAGE STRUCTURE

Below is a visualization of an HTML page structure:



Only the `<body>` area (the white area) is displayed by the browser.

THE <!DOCTYPE> DECLARATION

- The <!DOCTYPE> declaration helps the browser to **display a web page correctly**.
- There are **different document types** on the web.
- To display a document correctly, the browser must know **both type and version**.
- The doctype declaration is **not case sensitive**. All cases are acceptable:

- **HTML5**

<!DOCTYPE html>

- **HTML 4.01**

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

- **XHTML 1.0**

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

HTML TAGS

- HTML Headings

- HTML headings are defined with the **<h1>** to **<h6>** tags:

- `<h1>This is a heading</h1>`
- `<h2>This is a heading</h2>`
- `<h3>This is a heading</h3>`

- HTML Paragraphs

- HTML paragraphs are defined with the **<p>** tag:
 - `<p>This is a paragraph.</p>`

HTML TAGS

■ HTML Links

- HTML links are defined with the **<a>** tag:
 - `<a href="http://www.w3schools.com">This is a link</a>`

■ HTML Images

- HTML images are defined with the **** tag.
- The source file (**src**), alternative text (**alt**), and size (**width** and **height**) are provided as **attributes**:
 - ``

HTML ELEMENTS

- HTML elements are written with a **start** tag, with an **end** tag, with the **content** in between:
 - `<tagname>content</tagname>`
 - The HTML **element** is everything from the start tag to the end tag:

Start tag	Element content	End tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>		



Some HTML elements do not have an end tag.

NESTED HTML ELEMENTS

- TML elements can be nested (elements can contain elements).
 - All HTML documents consist of nested HTML elements.
 - This example contains 4 HTML elements:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
  </body>
</html>
```

HTML ATTRIBUTES

Attributes provide additional information about HTML elements.

- HTML elements can have attributes
- Attributes provide additional information about an element
- Attributes are **always specified in the start tag**
- Attributes come in **name/value pairs like: name="value"**

```
<!DOCTYPE html>  
<html lang="en-US">  
<body>  
<h1>My First Heading</h1>  
<p>My first paragraph.</p>  
</body>  
</html>
```

HTML ATTRIBUTES

■ Another example:

```
<p title="About W3Schools">
```

W3Schools is a web developer's site.

It provides tutorials and references covering many aspects of web programming, including HTML, CSS, JavaScript, XML, SQL, PHP, ASP, etc.

```
</p>
```



When you move the mouse over the element, the title will be displayed as a tooltip.

CSS - INTRODUCTION

- Cascading Style Sheets

- CSS is a stylesheet language that describes the presentation of an HTML (or XML) document.
- CSS describes how elements must be rendered on screen, on paper, or in other media.

```
body {  
    background-color: #d0e4fe;  
}  
  
h1 {  
    color: orange;  
    text-align: center;  
}  
  
p {  
    font-family: "Times New Roman";  
    font-size: 20px;  
}
```

Result:

My First CSS Example

This is a paragraph.

WHAT IS CSS?

- **CSS** stands for **C**ascading **S**tyle **S**heets
- CSS describes **how HTML elements are to be displayed on screen, paper, or in other media**
- **CSS saves a lot of work.** It can control the layout of multiple web pages all at once
- External stylesheets are stored in **CSS files**

EXAMPLE

Welcome to My Homepage

Use the menu to select different Stylesheets

Stylesheet 1

Stylesheet 2

Stylesheet 3

Stylesheet 4

No Stylesheet

Same Page Different Stylesheets

This is a demonstration of how different stylesheets can change the layout of your HTML page. You can change the layout of this page by selecting different stylesheets in the menu, or by selecting one of the following links:

[Stylesheet1](#), [Stylesheet2](#), [Stylesheet3](#), [Stylesheet4](#).

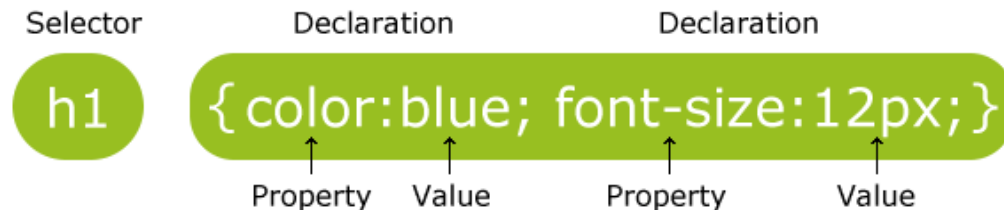
No Styles

This page uses DIV elements to group different sections of the HTML page. Click here to see how the page looks like with no stylesheet:

[No Stylesheet](#).

CSS SYNTAX

- A CSS rule-set consists of a selector and a declaration block:



- CSS selector
- The selector points to the HTML element you want to style.
- The declaration block contains one or more declarations separated by semicolons.
- Each declaration includes a CSS property name and a value, separated by a colon.
- A CSS declaration always ends with a semicolon, and declaration blocks are surrounded by curly braces.

THE ELEMENT SELECTOR

- The element selector selects elements based on the element name.
- You can select all `<p>` elements on a page like this (in this case, all `<p>` elements will be center-aligned, with a red text color):

```
p {  
    text-align: center;  
    color: red;  
}
```

Result:

Every paragraph will be affected by the style.

Me too!

And me!

THE ID SELECTOR

- The id selector uses the id attribute of an HTML element to select a specific element.
- The id of an element should be unique within a page, so the id selector is used to select one unique element!
- To select an element with a specific id, write a hash (#) character, followed by the id of the element.
- The style rule below will be applied to the HTML element with id="para1":

```
#para1 {  
    text-align: center;  
    color: red;  
}
```

REFRESH

■ HTML/ CSS/ SQL Quiz



PHP HELLO WORLD

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>Example</title>
  </head>
  <body>
    <?php
      echo "Hello World!";
    ?>
  </body>
</html>
```

BASIC PHP SYNTAX

- A PHP script can be placed anywhere in the document.
- A PHP script starts with **<?php** and ends with **?>**:

```
<?php  
// PHP code goes here  
?>
```

- The default file extension for PHP files is **".php"**.
- A PHP file normally contains HTML tags, and some PHP scripting code.

COMMENTS IN PHP

- Comments can be used to:
 - Let others understand what you are doing
 - Remind yourself of what you did

```
<!DOCTYPE html>
<html>
<body>
<?php
// This is a single-line comment

# This is also a single-line comment

/*
This is a multiple-lines comment block
that spans over multiple
lines
*/

// You can also use comments to leave out parts of a code line
$x = 5 /* + 15 */ + 5;
echo $x;
?>
</body>
</html>
```

PHP CASE SENSITIVITY

- In PHP
 - all keywords (e.g. if, else, while, echo, etc.)
 - classes
 - functions
 - user-defined functions are **NOT case-sensitive**

```
<?php  
ECHO "Hello World!<br>" ;  
echo "Hello World!<br>" ;  
Echo "Hello World!<br>" ;  
?>
```

However; all variable names are case-sensitive.

PHP 5 VARIABLES

- Creating (Declaring) PHP Variables

- In PHP, a variable starts with the \$ sign, followed by the name of the variable:

```
<?php  
$txt = "Hello world!";  
$x = 5;  
$y = 10.5;  
?>
```

- **Note:** When you assign a text value to a variable, put quotes around the value.
- **Note:** Unlike other programming languages, PHP has no command for declaring a variable. It is created the moment you first assign a value to it.



Think of variables as containers for storing data.

PHP VARIABLES

- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).
- Rules for PHP variables:
 - A variable starts with the \$ sign, followed by the name of the variable
 - A variable name must start with a letter or the underscore character
 - A variable name cannot start with a number
 - A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
 - Variable names are case-sensitive (\$age and \$AGE are two different variables)



Remember that PHP variable names are case-sensitive!

OUTPUT VARIABLES

```
<?php  
$txt = "W3Schools.com";  
echo "I love $txt!";  
?>
```

```
<?php  
$x = 5;  
$y = 4;  
echo $x + $y;  
?>
```

```
<?php  
$txt = "W3Schools.com";  
echo "I love " . $txt . "!";  
?>
```

PHP VARIABLE SCOPE

- In PHP, variables can be declared anywhere in the script.
- The scope of a variable is the part of the script where the variable can be referenced/used.
- PHP has three different variable scopes:
 - local
 - global
 - static

GLOBAL SCOPE

- A variable declared **outside** a function has a GLOBAL SCOPE and can only be accessed outside a function:

```
<?php
$x = 5; // global scope
function myTest() {
    // using x inside this function will generate an error
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();
echo "<p>Variable x outside function is: $x</p>";
?>
```

Result:

Variable x inside function is:

Variable x outside function is: 5

LOCAL SCOPE

- A variable declared **within** a function has a LOCAL SCOPE and can only be accessed within that function:

```
<?php
function myTest() {
    $x = 5; // local scope
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();

// using x outside the function will generate an error
echo "<p>Variable x outside function is: $x</p>";
?>
```

Result:

Variable x inside function is: 5

Variable x outside function is:

PHP THE GLOBAL KEYWORD

- The global keyword is used to access a global variable from within a function.

- ```
<?php
$x = 5;
$y = 10;
```

```
function myTest() {
 global $x, $y;
 $y = $x + $y;
}
```

```
myTest();
echo $y; // outputs 15
?>
```

```
<?php
$x = 5;
$y = 10;
```

```
function myTest() {
 $GLOBALS['y'] = $GLOBALS['x'] +
 $GLOBALS['y'];
}
```

```
myTest();
echo $y; // outputs 15
?>
```

- PHP also stores all global variables in an array called `$GLOBALS[index]`. The *index* holds the name of the variable. This array is also accessible from within functions and can be used to update global variables directly.

# PHP THE STATIC KEYWORD

- Normally, when a function is completed/executed, all of its variables are deleted. However, sometimes we want a local variable NOT to be deleted. We need it for a further job.

```
<?php
function myTest() {
 static $x = 0;
 echo $x;
 $x++;
}
myTest();
myTest();
myTest();
?>
```

Result:

0  
1  
2

- Then, each time the function is called, that variable will still have the information it contained from the last time the function was called.
- **Note:** The variable is still local to the function.

# PHP LOOPS

- Often when you write code, you want the same block of code to run over and over again in a row.
  - **while** - loops through a block of code as long as the specified condition is true
  - **do...while** - loops through a block of code once, and then repeats the loop as long as the specified condition is true
  - **for** - loops through a block of code a specified number of times
  - **foreach** - loops through a block of code for each element in an array

# THE PHP WHILE LOOP

- The while loop executes a block of code as long as the specified condition is true.

## Syntax

```
while (condition is true) {
 code to be executed;
}
```

```
<?php
```

```
$x = 1;
```

```
while($x <= 5) {
 echo "The number is: $x
";
 $x++;
}
?>
```

# THE PHP DO...WHILE LOOP

- The do...while loop will always execute the block of code once
- it will then check the condition, and repeat the loop while the specified condition is true.

## Syntax

```
do {
 code to be executed;
} while (condition is true);
```

```
<?php
```

```
$x = 1;
```

```
do {
```

```
 echo "The number is: $x
";
```

```
 $x++;
```

```
} while ($x <= 5);
```

```
?>
```

# THE PHP FOR LOOP

- The for loop is used when you know in advance how many times the script should run.

## Syntax

```
for (init counter; test counter; increment counter) {
 code to be executed;
}
```

## Parameters:

- *init counter*: **Initialize** the loop counter value
- *test counter*: Evaluated for each loop iteration. If it **evaluates to TRUE**, the loop continues. If it evaluates to FALSE, the loop ends.
- *increment counter*: **Increases** the loop counter value

```
<?php
for ($x = 0; $x <= 10; $x++) {
 echo "The number is: $x
";
}
?>
```

# THE PHP FOREACH LOOP

- The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

## Syntax

```
foreach ($array as $value) {
 code to be executed;
}
```

For every loop iteration, the **value of the current array element is assigned to \$value** and the **array pointer is moved by one**, until it reaches the last array element.

# THE PHP FOREACH LOOP

## Example:

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
 echo "$value
";
}
?>
```

Result:

```
red
green
blue
yellow
```

# PHP 5 ARRAYS

- An array stores multiple values in one single variable:

Example:

```
<?php
```

```
$cars = array("Volvo", "BMW", "Toyota");
```

```
echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . " .";
```

```
?>
```

# CREATE AN ARRAY IN PHP

- In PHP, the **array()** function is used to create an array:

```
array () ;
```

In PHP, there are three types of arrays:

- **Indexed arrays** - Arrays with a numeric index
- **Associative arrays** - Arrays with named keys
- **Multidimensional arrays** - Arrays containing one or more arrays

# PHP INDEXED ARRAYS

There are two ways to create indexed arrays:

- The index can be assigned automatically (**index always starts at 0**), like this:

```
$cars = array("Volvo", "BMW", "Toyota");
```

- or the index can be assigned manually:

```
$cars[0] = "Volvo";
$cars[1] = "BMW";
$cars[2] = "Toyota";
```

# GET THE LENGTH OF AN ARRAY - THE COUNT() FUNCTION

The **count()** function is used to return the length (the number of elements) of an array:

## Example:

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo count($cars);
?>
```

# LOOP THROUGH AN INDEXED ARRAY

- To loop through and print all the values of an indexed array, you could use a for loop, like this:

Example:

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
$arlength = count($cars);

for($x = 0; $x < $arlength; $x++) {
 echo $cars[$x];
 echo "
";
}
?>
```

# PHP ASSOCIATIVE ARRAYS

Associative arrays are arrays that use named keys that you assign to them.

- There are two ways to create an associative array:

```
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

or:

```
$age['Peter'] = "35";
```

```
$age['Ben'] = "37";
```

```
$age['Joe'] = "43";
```

- The named keys can then be used in a script:

# LOOP THROUGH AN ASSOCIATIVE ARRAY

- To loop through and print all the values of an associative array, you could use a foreach loop, like this:

Example:

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37",
"Joe"=>"43");

foreach($age as $x => $x_value) {
 echo "Key=" . $x . ", Value=" . $x_value;
 echo "
";
}
?>
```

# PHP 5 MULTIDIMENSIONAL ARRAYS

- A **multidimensional array** is an array containing one or more arrays.
- The **dimension of an array** indicates the number of indices you need to select an element.
- For a **two-dimensional array** you need two indices to select an element
- For a **three-dimensional array** you need three indices to select an element

# PHP - TWO-DIMENSIONAL ARRAYS

- A two-dimensional array is an array of arrays (a three-dimensional array is an array of arrays of arrays).

```
$cars = array
(
 array("Volvo",22,18) ,
 array("BMW",15,13) ,
 array("Saab",5,2) ,
 array("Land Rover",17,15)
);
```

| Name       | Stock | Sold |
|------------|-------|------|
| Volvo      | 22    | 18   |
| BMW        | 15    | 13   |
| Saab       | 5     | 2    |
| Land Rover | 17    | 15   |

# LOOPING THROUGH A 2 DIMENSIONAL ARRAY

```
<?php
```

```
for ($row = 0; $row < 4; $row++) {
 echo "<p>Row number $row</p>";
 echo "";
 for ($col = 0; $col < 3; $col++) {
 echo "".$cars[$row][$col]."";
 }
 echo "";
}
?>
```

# BRAIN-BREAK

## ■ What is MySQL?



# CONNECTING WITH A DATABASE

## PHP + MYSQL DATABASE SYSTEM

- PHP combined with MySQL are cross-platform (you can develop in Windows and serve on a Unix platform)

PHP 5 and later can work with a MySQL database using:

- **MySQLi extension** (the "i" stands for improved)
- **PDO (PHP Data Objects)**
  - Earlier versions of PHP used the MySQL extension. However, this extension was deprecated in 2012.

# PHP CONNECT TO MYSQL

Both MySQLi and PDO have their advantages:

- PDO will work on **12 different database systems**, where as MySQLi will only work with MySQL databases.
- So, if you have to **switch your project to use another database**, PDO makes the process easy. You **only have to change the connection string** and a few queries. With MySQLi, you will need to rewrite the entire code - queries included.
- Both are object-oriented, but **MySQLi also offers a procedural API**.
- Both support Prepared Statements. Prepared Statements protect from SQL injection, and are very important for web application security.

# OPEN A CONNECTION TO MYSQL

Example (MySQLi Object-Oriented):

```
<?php
```

```
$servername = "localhost";
```

```
$username = "username";
```

```
$password = "password";
```

```
// Create connection
```

```
$conn = new mysqli($servername, $username, $password);
```

```
// Check connection
```

```
if ($conn->connect_error) {
```

```
 die("Connection failed: " . $conn->connect_error);
```

```
}
```

```
echo "Connected successfully";
```

```
?>
```

# OPEN A CONNECTION TO MYSQL

## Example (MySQLi Procedural)

```
<?php
```

```
$servername = "localhost";
```

```
$username = "username";
```

```
$password = "password";
```

```
// Create connection
```

```
$conn = mysqli_connect($servername, $username, $password);
```

```
// Check connection
```

```
if (!$conn) {
```

```
 die("Connection failed: " . mysqli_connect_error());
```

```
}
```

```
echo "Connected successfully";
```

```
?>
```

# OPEN A CONNECTION TO MYSQL

## Example (PDO)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
try {
 $conn = new PDO("mysql:host=$servername;dbname=myDB",
$username, $password);
 // set the PDO error mode to exception
 $conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
 echo "Connected successfully";
}catch(PDOException $e){
 echo "Connection failed: " . $e->getMessage();
}
?>
```

# CLOSE THE CONNECTION

- Example (MySQLi Object-Oriented)
  - `$conn->close();`
- Example (MySQLi Procedural)
  - `mysqli_close($conn);`
- Example (PDO)
  - `$conn = null;`

# SELECT DATA FROM A MYSQL DATABASE

## Example (MySQLi Procedural)

```
$sql = "SELECT id, firstname, lastname FROM MyGuests";
$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) > 0) {
 // output data of each row
 while($row = mysqli_fetch_assoc($result)) {
 echo "id: " . $row["id"]. " - Name: " .
$row["firstname"]. " " . $row["lastname"]. "
";
 }
} else {
 echo "0 results";
}
```

# SUMMARY

- HTML, CSS and SQL Recap
- PHP Basic programming constructs – PHP Variables
- PHP Arrays
- Database Connectivity with PHP
- PHP WHILE loops

## NEXT LESSON

- GET/POST methods